



**LAPORAN PRAKTIKUM
P5 – STATEMENT PERULANGAN
(LOOPING)**

Nama : Muhammad Fadil Adiatma

NRP : 3226600139

Kelas : 1 TK C

Dosen Pengampu : Dr. Ir. Sigit Wasista M. Kom

PENDAHULUAN

5.1 Tujuan Praktikum

Tuliskan tujuan praktikum berdasarkan modul, misalnya:

- Menjelaskan konsep perulangan dan alasan penggunaannya dalam program.
- Membedakan karakteristik for, while, dan do-while berdasarkan posisi pemeriksaan kondisi.
- Menggunakan for untuk perulangan dengan pola penghitung yang jelas.
- Menggunakan while ketika kondisi perlu diperiksa sebelum badan perulangan dijalankan.
- Menggunakan do-while ketika badan perulangan harus dijalankan minimal satu kali.
- Menerapkan nested loop, break, dan continue pada kasus pemrograman sederhana.
- Menguji program dengan beberapa data masukan dan menjelaskan hasilnya.

5.2 Dasar Teori

Perulangan (looping) adalah mekanisme untuk menjalankan sekumpulan instruksi secara berulang selama kondisi tertentu terpenuhi. Perulangan menghindari penulisan kode yang sama berkali-kali dan sangat penting untuk pengolahan data, pencarian, perhitungan, serta pembentukan pola.

- **Looping:** Proses menjalankan suatu blok kode secara berulang selama kondisi tertentu terpenuhi.
- **Fungsi looping:** Mempermudah program untuk melakukan perintah berulang tanpa menulis kode yang sama berkali-kali.
- **for:** Perulangan yang biasanya digunakan ketika jumlah pengulangan sudah diketahui.
- **while:** Perulangan yang menjalankan kode selama kondisi bernilai benar (true). Kondisi diperiksa sebelum perulangan.
- **do-while:** Perulangan yang menjalankan kode **minimal satu kali**, kemudian memeriksa kondisi.
- **Nested loop:** Perulangan yang berada di dalam perulangan lainnya.
- **break:** Menghentikan perulangan secara langsung meskipun kondisi perulangan masih terpenuhi.
- **continue:** Melewati sisa kode pada iterasi saat ini dan melanjutkan ke iterasi berikutnya.

5.5 PERCOBAAN TERBIMBING

Percobaan 1 – Perulangan for

A. Tujuan

Memahami penggunaan perulangan while dalam bahasa pemrograman C

B. Source Code

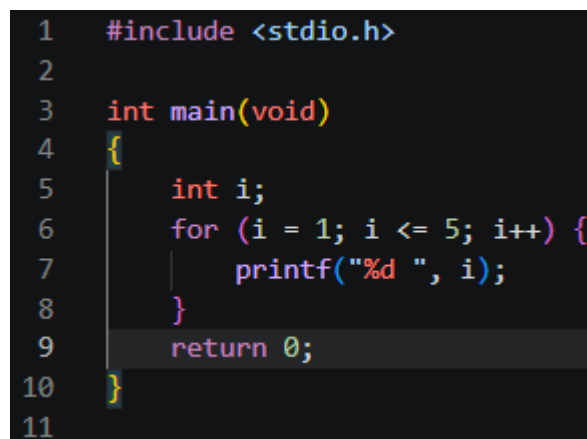
```
#include <stdio.h>

int main(void) {
    int i;
    for (i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    return 0; }
```

C. Algoritma

1. Mulai.
2. Inisialisasi variabel $i = 1$.
3. Periksa kondisi $i \leq 5$.
4. Jika benar, tampilkan nilai i .
5. Tambahkan i sebesar 1.
6. Kembali ke pemeriksaan kondisi.
7. Jika kondisi salah, program selesai.

D. Hasil Percobaan

A screenshot of a code editor showing a C program. The code is as follows:

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int i;
6      for (i = 1; i <= 5; i++) {
7          printf("%d ", i);
8      }
9      return 0;
10 }
11
```

The code is highlighted with a dark background and light text. The line numbers 1 through 11 are visible on the left side of the editor.

Screenshot hasil program:

1 2 3 4 5

E. Analisis

Program tersebut menggunakan perulangan **while** untuk menampilkan angka dari 1 sampai 5. Pada awal program, variabel *i* diberi nilai 1. Selanjutnya, kondisi *i* <= 5 diperiksa sebelum perulangan dijalankan. Jika kondisi terpenuhi, program menampilkan nilai *i* menggunakan `printf()`, kemudian nilai *i* ditambah 1 dengan `i++`. Proses tersebut terus berulang hingga nilai *i* menjadi 6, sehingga kondisi *i* <= 5 tidak terpenuhi dan perulangan berhenti. Dengan demikian, output yang dihasilkan adalah **1 2 3 4 5**.

Percobaan 2 – Struktur While

A. Tujuan

Memahami dan menerapkan struktur perulangan `while` pada bahasa pemrograman C serta mengetahui cara kerja perulangan berdasarkan kondisi yang telah ditentukan.

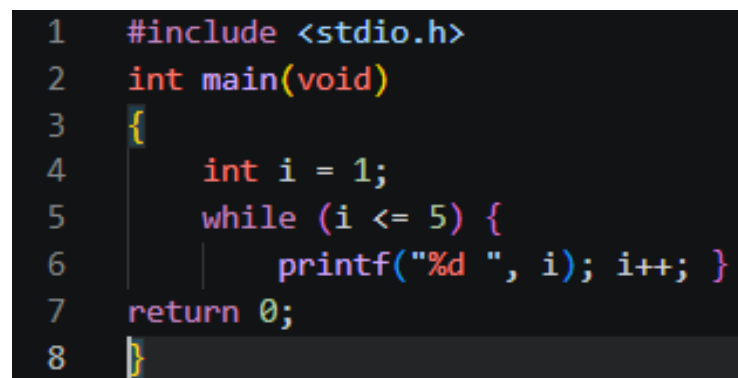
B. Source Code

```
#include <stdio.h>
int main(void)
{ int i = 1; while (i <= 5)
{ printf("%d ", i); i++; }
return 0; }
```

C. Algoritma

1. Mulai.
2. Deklarasikan variabel *i* dengan nilai awal 1.
3. Periksa kondisi apakah *i* <= 5.
4. Jika kondisi benar, tampilkan nilai *i*.
5. Tambahkan nilai *i* dengan 1.
6. Kembali ke langkah 3.
7. Jika kondisi salah, hentikan perulangan.
8. Selesai.

D. Hasil Percobaan



```
1  #include <stdio.h>
2  int main(void)
3  {
4      int i = 1;
5      while (i <= 5) {
6          printf("%d ", i); i++; }
7  return 0;
8  }
```

Screenshot hasil program

1 2 3 4 5

E. Analisis

Program menggunakan struktur perulangan while untuk menampilkan angka 1 sampai 5. Variabel *i* dimulai dari nilai 1 dan akan terus bertambah satu setiap perulangan menggunakan *i++*. Perulangan berlangsung selama kondisi *i* <= 5 terpenuhi. Setelah nilai *i* menjadi 6, kondisi tidak terpenuhi sehingga perulangan berhenti. Output yang dihasilkan adalah 1 2 3 4 5

Percobaan 3 – Struktur Do While

A. Tujuan

Memahami penggunaan struktur do-while serta membandingkan cara kerja perulangan while dan do-while dalam bahasa pemrograman C

B. Source Code.

```
#include <stdio.h>
int main(void)
{
    int i = 1;
    do { printf("%d ", i); i++; } while (i <= 5);
    return 0;
}
```

C. Algoritma

1. Mulai.
2. Deklarasikan variabel yang digunakan untuk perulangan.
3. Jalankan perintah di dalam blok do.
4. Perbarui nilai variabel perulangan.
5. Periksa kondisi pada while.
6. Jika kondisi benar, ulangi proses dari langkah 3.
7. Jika kondisi salah, hentikan perulangan.
8. Bandingkan hasil dan cara kerja while dengan do-while.
9. Selesai.

D. Hasil Percobaan

```
1  #include <stdio.h>
2  int main(void)
3  {
4  int i = 1;
5  do { printf("%d ", i); i++; } while (i <= 5);
6  return 0;
7  }
8  |
```

Screenshoot hasil program

1 2 3 4 5

E. Analisis

Struktur do-while merupakan perulangan yang menjalankan perintah terlebih dahulu, kemudian memeriksa kondisi. Oleh karena itu, blok kode pada do-while akan dijalankan minimal satu kali, meskipun kondisi awalnya salah. Berbeda dengan while, kondisi diperiksa terlebih dahulu sehingga blok kode dapat tidak dijalankan sama sekali jika kondisi awal tidak terpenuhi. Dengan demikian, perbedaan utama keduanya terletak pada urutan pemeriksaan kondisi dan pelaksanaan perintah.

Percobaan 4 - Perbandingan

Jenis	Pemeriksaan kondisi	Minimal eksekusi badan	Kapan cocok digunakan?
for	Sebelum setiap iterasi	0 kali	Jumlah/pola iterasi sudah jelas atau menggunakan counter.
while	Sebelum setiap iterasi	0 kali	Perulangan bergantung pada kondisi yang harus diperiksa lebih dahulu.
do-while	Sesudah setiap iterasi	1 kali	Badan program harus dijalankan minimal sekali, misalnya menu dan validasi input.

Percobaan Terbimbing

Percobaan 1 - Triangular

A. Tujuan

Memahami penggunaan perulangan for dalam bahasa pemrograman C untuk menghitung jumlah bilangan triangular dari 1 sampai 200 serta memahami proses penjumlahan secara berulang.

B. Source Code

```
#include <stdio.h>
int main(void) {
    int n, jumlah = 0;
    for (n = 1; n <= 200; n++) {
        jumlah += n;
    }
    printf("Jumlah 200 triangular = %d\n", jumlah);
    return 0;
}
```

C. Logaritma

1. Mulai.
2. Deklarasikan variabel n dan jumlah.
3. Berikan nilai awal jumlah = 0.
4. Inisialisasi n = 1.
5. Periksa kondisi n <= 200.
6. Jika kondisi benar, tambahkan nilai n ke jumlah.
7. Tambahkan nilai n dengan 1.
8. Ulangi langkah 5 sampai nilai n lebih dari 200.
9. Tampilkan hasil jumlah.
10. Selesai.

D. Hasil Percobaan

```
1  #include <stdio.h>
2  int main(void) {
3      int n, jumlah = 0;
4      for (n = 1; n <= 200; n++) {
5          jumlah += n;
6      }
7      printf("Jumlah 200 triangular = %d\n", jumlah);
8      return 0;
9  }
```

Screenshot hasil

```
Jumlah 200 triangular = 20100
```

E. Analisis

Program menggunakan perulangan for untuk menghitung jumlah bilangan dari 1 sampai 200. Variabel jumlah diawali dengan nilai 0, kemudian setiap nilai n ditambahkan ke dalam jumlah selama nilai n masih kurang atau sama dengan 200. Setelah perulangan selesai, diperoleh hasil penjumlahan 20100, sehingga output program adalah "Jumlah 200 triangular = 20100".

Percobaan 2 – Nested Loop

A. Tujuan

memahami penggunaan nested loop atau perulangan bersarang dalam bahasa pemrograman C serta mengetahui cara membuat pola menggunakan dua perulangan for.

B. Source Code

```
#include <stdio.h>
int main(void)
{
    int i, j;
    for (i = 1; i <= 5; i++) {
        for (j = 1; j <= i; j++) {
            printf("* ");
        }
    }
}
```

```
} printf("\n"); }  
return 0;  
}
```

C. Algoritma

1. Mulai.
2. Deklarasikan variabel i dan j.
3. Jalankan perulangan luar i dari 1 sampai 5.
4. Untuk setiap nilai i, jalankan perulangan dalam j dari 1 sampai nilai i.
5. Tampilkan karakter * pada setiap perulangan dalam.
6. Setelah perulangan dalam selesai, pindah ke baris baru.
7. Ulangi hingga perulangan luar selesai.
8. Selesai.

D. Hasil Percobaan

```
1  #include <stdio.h>  
2  int main(void) {  
3      int i, j;  
4      for (i = 1; i <= 5; i++) {  
5          for (j = 1; j <= i; j++) {  
6              printf("* ");  
7          }  
8          printf("\n");  
9      }  
10     return 0;  
11 }
```

Screenshot Hasil

```
1  #include <stdio.h>  
2  int main(void) {  
3      int i, j;  
4      for (i = 1; i <= 5; i++) {  
5          for (j = 1; j <= i; j++) {  
6              printf("* ");  
7          }  
8          printf("\n");  
9      }  
10     return 0;  
11 }
```



```
*  
* *  
* * *  
* * * *  
* * * * *
```

E. Analisis

Program menggunakan nested loop, yaitu perulangan for yang berada di dalam perulangan for lainnya. Perulangan luar menggunakan variabel i untuk menentukan jumlah baris, sedangkan perulangan dalam menggunakan variabel j untuk menentukan jumlah tanda * pada setiap baris. Setiap baris menghasilkan jumlah tanda * yang bertambah satu, sehingga membentuk pola segitiga

Percobaan 3 – Break

A. Tujuan

memahami penggunaan break dalam perulangan for serta mengetahui cara menghentikan perulangan sebelum kondisi perulangan selesai.

B. Source Code

```
#include <stdio.h>
int main(void)
{
    int i; for (i = 1; i <= 10; i++) {
        printf("Proses ke-%d\n", i); if (i == 5) {
            printf("Perulangan dihentikan.\n");
            break; }
        }
    return 0;
}
```

C. Algoritma

Mulai.

1. Deklarasikan variabel i.
2. Jalankan perulangan for dengan nilai i dari 1 sampai 10.
3. Tampilkan teks "Proses ke-" dan nilai i.
4. Periksa apakah nilai i sama dengan 5.
5. Jika benar, tampilkan pesan "Perulangan dihentikan."
6. Jalankan break untuk menghentikan perulangan.
7. Jika kondisi tidak terpenuhi, lanjutkan ke perulangan berikutnya.
8. Selesai.

D. Hasil Percobaan

```
1  #include <stdio.h>
2  int main(void) {
3      int i;
4      for (i = 1; i <= 10; i++) {
5          printf("Proses ke-%d\n", i);
6          if (i == 5) {
7              printf("Perulangan dihentikan.\n");
8              break;
9          }
10     }
11     return 0;
12 }
13
```

Screenshot Hasil

```
Proses ke-1
Proses ke-2
Proses ke-3
Proses ke-4
Proses ke-5
Perulangan dihentikan.
```

E. Analisis

Program menggunakan perulangan for yang seharusnya berjalan dari nilai 1 sampai 10. Namun, di dalam perulangan terdapat kondisi if ($i == 5$) yang akan menjalankan perintah break ketika nilai i mencapai 5. Akibatnya, perulangan berhenti pada proses ke-5 dan tidak melanjutkan ke proses berikutnya. Output terakhir yang ditampilkan adalah "Perulangan dihentikan.", sehingga break berfungsi untuk menghentikan perulangan secara langsung.

Percobaan 4 – Continue

A. Tujuan

Memahami penggunaan continue pada perulangan for serta mengetahui cara melewati satu proses perulangan tanpa menghentikan keseluruhan perulangan.

B. Source Code

```
#include <stdio.h>
int main(void)
{
    int i; for (i = 1; i <= 10; i++) {
        if (i == 5) {
            continue; }
        printf("%d ", i);
    }
    return 0;
}
```

C. Algoritma

1. Mulai.
2. Deklarasikan variabel i .
3. Jalankan perulangan for dari $i = 1$ sampai $i \leq 10$.
4. Periksa apakah $i == 5$.
5. Jika benar, jalankan perintah continue untuk melewati proses tersebut.
6. Jika salah, tampilkan nilai i .
7. Lanjutkan ke perulangan berikutnya.
8. Ulangi hingga nilai i lebih dari 10.
9. Selesai.

D. Hasil Percobaan

```
1  #include <stdio.h>
2  int main(void) {
3      int i;
4      for (i = 1; i <= 10; i++) {
5          if (i == 5) {
6              continue;
7          }
8          printf("%d ", i);
9      }
10     return 0;
11 }
```

Screenshot Hasil



E. Analisis

Program menggunakan perulangan `for` untuk menampilkan angka dari 1 sampai 10. Ketika nilai `i` mencapai 5, kondisi `i == 5` terpenuhi sehingga perintah `continue` dijalankan. Perintah tersebut menyebabkan proses pada iterasi ke-5 dilewati dan program langsung melanjutkan ke iterasi berikutnya. Oleh karena itu, angka 5 tidak ditampilkan, sedangkan angka lainnya tetap ditampilkan. Output program adalah 1 2 3 4 6 7 8 9 10.

5.6 LATIHAN TERBIMBING

1. Latihan `for` – Menjumlahkan Bilangan 1 sampai `n`

A. Permasalahan

Hitung jumlah bilangan 1 sampai `n`.

B. Algoritma

1. Input nilai `n`.
2. Inisialisasi `jumlah = 0`.
3. Lakukan perulangan dari 1 sampai `n`.
4. Tambahkan setiap nilai ke `jumlah`.
5. Tampilkan hasil.

C. Source Code

```
#include <stdio.h>
int main(void)
{
    int n, i, jumlah = 0;
    printf("Masukkan n: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        jumlah += i;
    }
    printf("Jumlah = %d\n", jumlah);
    return 0;
}
```

D. Pengujian

NO	Input	Output	Keterangan
1	5	15	Benar
2	10	55	Benar
3	15	120	Benar

E. Screenshot

```
1  #include <stdio.h>
2  int main(void) {
3      int n, i, jumlah = 0;
4      printf("Masukkan n: ");
5      scanf("%d", &n);
6      for (i = 1; i <= n; i++) {
7          jumlah += i;
8      }
9      printf("Jumlah = %d\n", jumlah);
10     return 0;
11 }
```

Screenshot hasil

```
Masukkan n: 10
Jumlah = 55
```

F. Analisis

Program menggunakan perulangan **for** untuk menjumlahkan bilangan dari 1 sampai n. Nilai n dimasukkan oleh pengguna, kemudian setiap bilangan dari 1 hingga n ditambahkan ke variabel jumlah. Setelah perulangan selesai, program menampilkan hasil penjumlahan. Dengan demikian, program dapat menghitung jumlah bilangan secara otomatis sesuai nilai n yang dimasukkan.

2. Latihan **while** – Validasi Nilai

A. Permasalahan

bagaimana membuat program yang dapat **memvalidasi nilai yang dimasukkan pengguna** agar berada dalam rentang 0 sampai 100

B. Algoritma

1. Mulai.
2. Deklarasikan variabel `nilai`.
3. Minta pengguna memasukkan nilai 0–100.

4. Periksa apakah `nilai < 0` atau `nilai > 100`.
5. Jika kondisi benar, tampilkan pesan "Nilai tidak valid" dan minta pengguna memasukkan nilai kembali.
6. Ulangi pemeriksaan hingga nilai berada pada rentang 0–100.
7. Tampilkan nilai yang telah diterima.
8. Selesai.

C. Source Code

```
#include <stdio.h>
int main(void)
{
    int nilai;
    printf("Masukkan nilai 0-100: ");
    scanf("%d", &nilai);
    while (nilai < 0 || nilai > 100) {
        printf("Nilai tidak valid. Masukkan lagi: ");
        scanf("%d", &nilai);
    }
    printf("Nilai diterima = %d\n", nilai);
    return 0;
}
```

D. Pengujian

No	Input	Output	Keterangan
1	30	30	Benar
2	99	99	Benar
3	101	Tidak valid	Benar

E. Screenshot

```
1  #include <stdio.h>
2  int main(void) {
3      int nilai;
4      printf("Masukkan nilai 0-100: ");
5      scanf("%d", &nilai);
6      while (nilai < 0 || nilai > 100) {
7          printf("Nilai tidak valid. Masukkan lagi: ");
8          scanf("%d", &nilai);
9      }
10     printf("Nilai diterima = %d\n", nilai);
11     return 0;
12 }
```

```
Masukkan nilai 0-100: 101
Nilai tidak valid. Masukkan lagi: 111
Nilai tidak valid. Masukkan lagi: 200
Nilai tidak valid. Masukkan lagi: 20
Nilai diterima = 20
```

E. Analisis

Program menggunakan perulangan **while** untuk melakukan validasi nilai. Jika nilai yang dimasukkan kurang dari 0 atau lebih dari 100, program akan menampilkan pesan bahwa nilai tidak valid dan meminta input kembali. Perulangan berhenti ketika nilai berada dalam rentang 0 sampai 100. Setelah itu, program menampilkan nilai yang diterima. Dengan demikian, program memastikan bahwa input yang digunakan sesuai dengan batas nilai yang telah ditentukan.

3. Latihan **do-while** – Menu Sederhana

A. Permasalahan

Pembuatan menu sederhana yang dapat menerima pilihan pengguna dan menjalankan perintah sesuai pilihan tersebut.

B. Algoritma

1. Mulai.
2. Deklarasikan variabel `pilihan`.
3. Tampilkan menu pilihan.
4. Minta pengguna memasukkan pilihan.
5. Jika pilihan 1, tampilkan pesan **"Praktikum Pemrograman Dasar"**.
6. Jika pilihan 2, tampilkan angka **1 2 3 4 5**.
7. Jika pilihan bukan 0, 1, atau 2, tampilkan pesan **"Pilihan tidak tersedia."**
8. Periksa apakah pilihan tidak sama dengan 0.
9. Jika benar, ulangi proses dari langkah 3.
10. Jika pilihan 0, hentikan perulangan.
11. Selesai.

C. Source Code

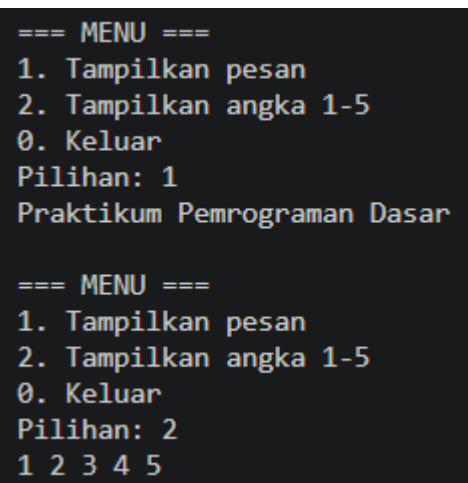
```
#include <stdio.h>
int main(void)
{
    int pilihan; do {
        printf("\n=== MENU ===\n");
        printf("1. Tampilkan pesan\n");
        printf("2. Tampilkan angka 1-5\n");
        printf("0. Keluar\n");
        printf("Pilihan: ");
        scanf("%d", &pilihan);
        if (pilihan == 1)
            printf("Praktikum Pemrograman Dasar\n");
        else if (pilihan == 2)
            printf("1 2 3 4 5\n");
        else if (pilihan != 0)
            printf("Pilihan tidak tersedia.\n"); }
    while (pilihan != 0);
    return 0; }
```

D. Pengujian

No	Input	Output	Keterangan
1	1	Praktikum pemrograman dasar	Benar
2	2	1 2 3 4 5	Benar
3	0		Benar

E. Screenshot

```
1  #include <stdio.h>
2  int main(void) {
3      int pilihan;
4      do {
5          printf("\n=== MENU ===\n");
6          printf("1. Tampilkan pesan\n");
7          printf("2. Tampilkan angka 1-5\n");
8          printf("0. Keluar\n");
9          printf("Pilihan: ");
10         scanf("%d", &pilihan);
11         if (pilihan == 1)
12             printf("Praktikum Pemrograman Dasar\n");
13         else if (pilihan == 2)
14             printf("1 2 3 4 5\n");
15         else if (pilihan != 0)
16             printf("Pilihan tidak tersedia.\n");
17         } while (pilihan != 0);
18         return 0;
19     }
```



F. Analisis

Program menggunakan struktur `do-while` untuk membuat menu yang dapat dijalankan berulang kali. Menu akan ditampilkan terlebih dahulu dan pengguna memasukkan pilihan. Pilihan 1 akan menampilkan pesan, pilihan 2 menampilkan angka 1 sampai 5, sedangkan pilihan lainnya akan menampilkan pesan bahwa pilihan tidak tersedia. Perulangan terus berjalan selama pilihan tidak sama dengan 0. Ketika pengguna memilih 0, kondisi `pilihan != 0` menjadi salah sehingga program berhenti. Struktur `do-while` sesuai digunakan karena menu harus ditampilkan setidaknya satu kali.

4. Latihan Nested Loop – Pola Segitiga

A. Permasalahan

bagaimana membuat program menggunakan nested loop untuk menghasilkan pola segitiga dari karakter * sebanyak 5 baris, dengan jumlah karakter yang bertambah pada setiap baris.

B. Algoritma

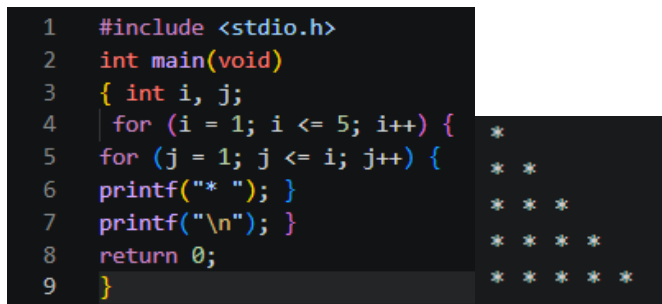
1. Mulai.
2. Deklarasikan variabel `i` dan `j`.
3. Jalankan perulangan luar `i` dari 1 sampai 5.

4. Jalankan perulangan dalam j dari 1 sampai nilai i.
5. Tampilkan karakter *.
6. Setelah perulangan dalam selesai, pindah ke baris baru.
7. Ulangi hingga perulangan luar selesai.
8. Selesai.

C. Source Code

```
#include <stdio.h>
int main(void)
{ int i, j;
  for (i = 1; i <= 5; i++) {
    for (j = 1; j <= i; j++) {
      printf("* ");
    }
    printf("\n");
  }
  return 0; }
```

D. Screenshot



```
1  #include <stdio.h>
2  int main(void)
3  { int i, j;
4    for (i = 1; i <= 5; i++) {
5      for (j = 1; j <= i; j++) {
6        printf("* ");
7      }
8      printf("\n");
9    }
10   return 0; }
```

*
* *
* * *
* * * *
* * * * *

E. Analisis

Program menggunakan nested loop, yaitu perulangan yang berada di dalam perulangan lainnya. Perulangan luar (i) menentukan jumlah baris, sedangkan perulangan dalam (j) menentukan jumlah karakter * pada setiap baris. Setiap baris memiliki jumlah karakter yang bertambah satu, sehingga menghasilkan pola segitiga sebanyak 5 baris.

Fungsi loop luar (i): Mengatur jumlah baris yang akan dibuat. Pada program, i berjalan dari 1 sampai 5 sehingga menghasilkan 5 baris.

Fungsi loop dalam (j): Mengatur jumlah karakter * yang dicetak pada setiap baris. Nilai j berjalan dari 1 sampai nilai i.

Hubungan antara baris dan kolom: Nilai i menentukan baris, sedangkan nilai j menentukan jumlah kolom atau * pada baris tersebut. Semakin besar nilai i, semakin banyak * yang dicetak. Contohnya, baris ke-1 memiliki 1 *, baris ke-2 memiliki 2 *, hingga baris ke-5 memiliki 5 *.

5.7 TUGAS PRAKTIKUM

Modul memberikan **10 tugas praktikum**, dan setiap program harus disertai **source code**, **contoh input-output**, serta **penjelasan algoritma**.

Tugas 1 – Matriks Identitas

Soal:

Buat program untuk mencetak matriks identitas berukuran $n \times n$.

Algoritma:

1. Mulai.
2. Masukkan ukuran matriks n .
3. Gunakan perulangan i untuk menentukan baris.
4. Gunakan perulangan j untuk menentukan kolom.
5. Jika $i == j$, cetak 1.
6. Jika $i != j$, cetak 0.
7. Ulangi hingga seluruh matriks tercetak.
8. Selesai.

Source Code:

```
1  #include <stdio.h>
2
3  int main(void) {
4      int n, i, j;
5
6      printf("Masukkan ukuran matriks: ");
7      scanf("%d", &n);
8
9      for (i = 0; i < n; i++) {
10         for (j = 0; j < n; j++) {
11             if (i == j)
12                 printf("1 ");
13             else
14                 printf("0 ");
15         }
16         printf("\n");
17     }
18
19     return 0;
20 }
```

Input:

$n = 3$

Output

```
Masukkan ukuran matriks: 3
1 0 0
0 1 0
0 0 1
```

Analisis:

Program menggunakan nested loop untuk mencetak matriks berukuran $n \times n$. Perulangan i digunakan untuk menentukan baris dan j untuk menentukan kolom. Kondisi $i == j$ digunakan untuk mencetak angka 1 pada posisi diagonal utama, sedangkan posisi lainnya dicetak 0. Dengan demikian, program menghasilkan matriks identitas sesuai ukuran yang dimasukkan pengguna.

Tugas 2 – Tabel Perkalian 1–10

Soal:

Buat program menggunakan `nested loop` untuk menampilkan tabel perkalian 1 sampai 10.

Algoritma:

1. Mulai.
2. Deklarasikan variabel `i` dan `j`.
3. Gunakan perulangan luar `i` dari 1 sampai 10.
4. Gunakan perulangan dalam `j` dari 1 sampai 10.
5. Kalikan `i` dengan `j`.
6. Tampilkan hasil perkalian.
7. Ulangi hingga seluruh tabel perkalian selesai.
8. Selesai

Source Code:

```
1  #include <stdio.h>
2
3  int main(void) {
4      int i, j;
5
6      for (i = 1; i <= 10; i++) {
7          for (j = 1; j <= 10; j++) {
8              printf("%4d", i * j);
9          }
10         printf("\n");
11     }
12
13     return 0;
14 }
```

Output:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

Analisis:

Program menggunakan `nested loop` untuk membuat tabel perkalian 1 sampai 10. Perulangan luar `i` menentukan baris, sedangkan perulangan dalam `j` menentukan kolom. Setiap elemen tabel diperoleh dari hasil perkalian `i * j`. Perulangan terus dilakukan hingga `i` dan `j` mencapai 10 sehingga menghasilkan tabel perkalian lengkap dari 1 sampai 10.

Tugas 3 – Membalikkan Digit

Contoh:

Input : 1234

Output : 4321

Algoritma:

1. Mulai.
2. Masukkan sebuah bilangan.
3. Ambil digit terakhir menggunakan operasi % 10.
4. Tambahkan digit tersebut ke variabel balik.
5. Hilangkan digit terakhir menggunakan operasi / 10.
6. Ulangi langkah 3–5 selama bilangan masih lebih dari 0.
7. Tampilkan bilangan yang telah dibalik.
8. Selesai.

Source Code:

```
1  #include <stdio.h>
2
3  int main(void) {
4      int angka, balik = 0, digit;
5
6      printf("Masukkan angka: ");
7      scanf("%d", &angka);
8
9      while (angka > 0) {
10         digit = angka % 10;
11         balik = balik * 10 + digit;
12         angka = angka / 10;
13     }
14
15     printf("Hasil pembalikan = %d\n", balik);
16
17     return 0;
18 }
```

Pengujian:

Input	Output	Keterangan
1234	4321	Benar
5678	8765	Benar
123456	654321	Benar

Analisis:

Program menggunakan perulangan while untuk membalikkan digit suatu bilangan. Operator % 10 digunakan untuk mengambil digit terakhir, sedangkan / 10 digunakan untuk menghilangkan digit terakhir. Digit yang diperoleh kemudian disusun kembali dari belakang menggunakan $\text{balik} = \text{balik} * 10 + \text{digit}$. Proses berlangsung hingga seluruh digit habis, sehingga bilangan 1234 menjadi 4321.

Tugas 4 – Konversi Suhu

Buat tabel konversi:

- Celsius → Fahrenheit
- Celsius → Reamur
- Celsius → Kelvin

Algoritma

1. Mulai.
2. Masukkan suhu dalam Celsius.
3. Hitung Fahrenheit dengan rumus $(\text{Celsius} \times 9/5) + 32$.
4. Hitung Reamur dengan rumus $\text{Celsius} \times 4/5$.
5. Hitung Kelvin dengan rumus $\text{Celsius} + 273,15$.
6. Tampilkan hasil konversi Celsius, Fahrenheit, Reamur, dan Kelvin.
7. Selesai.

Source Code:

```
1  #include <stdio.h>
2
3  int main(void) {
4      float celsius, fahrenheit, reamur, kelvin;
5
6      printf("Masukkan suhu Celsius: ");
7      scanf("%f", &celsius);
8
9      fahrenheit = (celsius * 9 / 5) + 32;
10     reamur = celsius * 4 / 5;
11     kelvin = celsius + 273.15;
12
13     printf("Celsius      : %.2f C\n", celsius);
14     printf("Fahrenheit   : %.2f F\n", fahrenheit);
15     printf("Reamur       : %.2f R\n", reamur);
16     printf("Kelvin        : %.2f K\n", kelvin);
17
18     return 0;
19 }
```

Pengujian:

Input = 100

Output :

```
Masukkan suhu Celsius: 100
Celsius      : 100.00 C
Fahrenheit   : 212.00 F
Reamur       : 80.00 R
Kelvin       : 373.15 K
```

Analisis:

Perhitungan dilakukan menggunakan rumus konversi masing-masing. Hasil ditampilkan dengan dua angka di belakang koma menggunakan format %.2f . Program dapat menghasilkan nilai konversi sesuai dengan suhu Celsius yang dimasukkan pengguna.

Tugas 5 – Jumlah dan Rata-rata n Nilai

Algoritma

1. Mulai.
2. Masukkan jumlah data n.
3. Inisialisasi jumlah = 0.
4. Jalankan perulangan dari data ke-1 sampai data ke-n.
5. Masukkan nilai.
6. Tambahkan nilai ke variabel jumlah.
7. Ulangi hingga seluruh data dimasukkan.
8. Hitung rata-rata dengan rumus jumlah / n.
9. Tampilkan jumlah dan rata-rata.
10. Selesai.

Source Code:

```
1  #include <stdio.h>
2
3  int main(void) {
4      int n, i;
5      float nilai, jumlah = 0, rata_rata;
6
7      printf("Masukkan jumlah data: ");
8      scanf("%d", &n);
9
10     for (i = 1; i <= n; i++) {
11         printf("Masukkan nilai ke-%d: ", i);
12         scanf("%f", &nilai);
13         jumlah += nilai;
14     }
15
16     rata_rata = jumlah / n;
17
18     printf("Jumlah = %.2f\n", jumlah);
19     printf("Rata-rata = %.2f\n", rata_rata);
20
21     return 0;
22 }
```

Pengujian:

No	Jumlah data	Data	Jumlah	Rata-rata
1	3	2, 3, 4	9	3
2	3	70, 80, 90	240	80
3	5	15, 20, 25, 30, 35,	125	25

Analisis

Program menggunakan perulangan for untuk menerima beberapa nilai sesuai jumlah data yang dimasukkan. Setiap nilai dijumlahkan ke dalam variabel jumlah. Setelah semua data dimasukkan, program menghitung rata-rata dengan membagi jumlah nilai dengan banyaknya data. Hasil pengujian menunjukkan bahwa program dapat menghitung jumlah dan rata-rata nilai dengan benar.

Tugas 6 – Jumlah Digit Bilangan

Algoritma

1. Mulai.
2. Masukkan sebuah bilangan.
3. Inisialisasi jumlah = 0.
4. Ambil digit terakhir menggunakan operasi % 10.
5. Tambahkan digit tersebut ke jumlah.
6. Hilangkan digit terakhir menggunakan / 10.
7. Ulangi langkah 4–6 sampai seluruh digit habis.
8. Tampilkan hasil jumlah digit.
9. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main(void) {
4      int angka, digit, jumlah = 0;
5
6      printf("Masukkan bilangan: ");
7      scanf("%d", &angka);
8
9      while (angka > 0) {
10         digit = angka % 10;
11         jumlah += digit;
12         angka = angka / 10;
13     }
14
15     printf("Jumlah digit = %d\n", jumlah);
16
17     return 0;
18 }
```

Pengujian

Input	Output
3255	15
9999	36
1234	10

Analisis

Program digunakan untuk menghitung jumlah seluruh digit dari suatu bilangan. Perulangan while mengambil digit terakhir menggunakan operator % 10, kemudian digit tersebut ditambahkan ke variabel jumlah. Setelah itu, digit terakhir dihilangkan menggunakan pembagian / 10. Proses diulang sampai seluruh digit habis. Contohnya, bilangan 3255 dihitung menjadi $3 + 2 + 5 + 5 = 15$.

Tugas 7 – Tabel Faktorial

Algoritma

1. Mulai.
2. Deklarasikan variabel n, i, dan faktorial.
3. Jalankan perulangan n dari 1 sampai 10.
4. Atur faktorial = 1.
5. Jalankan perulangan i dari 1 sampai n.
6. Kalikan faktorial dengan i.
7. Ulangi hingga i mencapai n.
8. Tampilkan nilai n, n!, dan hasil faktorial.
9. Ulangi hingga n mencapai 10.
10. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main(void) {
4      int n, i;
5      int faktorial = 1;
6
7      printf("n    n!        Hasil\n");
8
9      for (n = 1; n <= 10; n++) {
10         faktorial = 1;
11
12         for (i = 1; i <= n; i++) {
13             faktorial *= i;
14         }
15
16         printf("%d    %d!        %d\n", n, n, faktorial);
17     }
18
19     return 0;
20 }
```

Output

n	n!	Hasil
1	1!	1
2	2!	2
3	3!	6
4	4!	24
5	5!	120
6	6!	720
7	7!	5040
8	8!	40320
9	9!	362880
10	10!	3628800

Analisis

Program menggunakan nested loop untuk menghitung dan menampilkan 10 faktorial pertama. Perulangan luar digunakan untuk menentukan nilai n dari 1 sampai 10, sedangkan perulangan dalam menghitung faktorial dengan mengalikan bilangan dari 1 hingga n. Variabel faktorial diatur kembali menjadi 1 pada setiap perulangan agar perhitungan setiap nilai tidak tercampur. Hasil akhirnya adalah nilai faktorial dari 1! sampai 10! yang ditampilkan dalam bentuk tabel.

Tugas 8 – Minimum, Maksimum, dan Rata-rata

Algoritma

1. Mulai.
2. Masukkan jumlah data n.
3. Inisialisasi variabel jumlah = 0.
4. Masukkan nilai pertama dan jadikan sebagai nilai minimum dan maksimum.
5. Masukkan nilai berikutnya.
6. Bandingkan nilai dengan minimum. Jika lebih kecil, jadikan nilai tersebut sebagai minimum.
7. Bandingkan nilai dengan maksimum. Jika lebih besar, jadikan nilai tersebut sebagai maksimum.
8. Tambahkan setiap nilai ke jumlah.
9. Ulangi langkah 5–8 sampai seluruh data selesai dimasukkan.
10. Hitung rata-rata dengan rumus jumlah / n.
11. Tampilkan nilai minimum, maksimum, dan rata-rata.
12. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main(void) {
4      int n, i;
5      float nilai, minimum, maksimum, jumlah = 0, rata_rata;
6      printf("Masukkan jumlah data: ");
7      scanf("%d", &n);
8      for (i = 1; i <= n; i++) {
9          printf("Masukkan nilai ke-%d: ", i);
10         scanf("%f", &nilai);
11
12         if (i == 1) {
13             minimum = nilai;
14             maksimum = nilai;
15         } else {
16             if (nilai < minimum)
17                 minimum = nilai;
18
19             if (nilai > maksimum)
20                 maksimum = nilai;
21         }
22         jumlah += nilai;
23     }
24     rata_rata = jumlah / n;
25     printf("\nNilai Minimum = %.2f\n", minimum);
26     printf("Nilai Maksimum = %.2f\n", maksimum);
27     printf("Rata-rata = %.2f\n", rata_rata);
28     return 0;
29 }
```

Pengujian

No	Jumlah data	Data	Min	Max	Rata-rata
1	3	10,20,30	10	30	20
2	3	1,2,3	1	3	2
3	5	70,65,80,90,95	65	95	80

Analisis

Program menggunakan perulangan for untuk menerima sejumlah data nilai siswa. Setiap nilai dibandingkan untuk menentukan nilai minimum dan maksimum, sedangkan seluruh nilai dijumlahkan untuk menghitung rata-rata. Nilai pertama digunakan sebagai nilai awal minimum dan maksimum, kemudian nilai berikutnya dibandingkan dengan keduanya. Setelah seluruh data selesai diproses, program menampilkan nilai minimum, maksimum, dan rata-rata.

Tugas 9 – break

Algoritma

1. Mulai.
2. Siapkan data {10, 25, 30, 45, 50}.
3. Masukkan nilai target yang ingin dicari.
4. Lakukan perulangan menggunakan for untuk memeriksa setiap data.
5. Bandingkan setiap data dengan nilai target.
6. Jika target ditemukan, tampilkan posisi data dan jalankan break untuk menghentikan perulangan.
7. Jika seluruh data telah diperiksa dan target tidak ditemukan, tampilkan pesan "Target tidak ditemukan."
8. Selesai.

Source Code

```
2
3  int main(void) {
4      int data[] = {10, 25, 30, 45, 50};
5      int n = 5;
6      int target, i;
7      int ditemukan = 0;
8
9      printf("Masukkan nilai target: ");
10     scanf("%d", &target);
11
12     for (i = 0; i < n; i++) {
13         if (data[i] == target) {
14             ditemukan = 1;
15             printf("Target ditemukan pada data ke-%d.\n", i + 1);
16             break;
17         }
18     }
19
20     if (ditemukan == 0) {
21         printf("Target tidak ditemukan.\n");
22     }
23
24     return 0;
25 }
```

Output

No	Nilai target	Output
1	90	Tidak ditemukan
2	50	Ditemukan pada data ke 5
3	10	Ditemukan pada data ke 1

Analisis

Program menggunakan perulangan for untuk mencari nilai target di dalam array. Jika nilai target ditemukan, variabel ditemukan diubah menjadi 1 dan perintah break digunakan untuk menghentikan perulangan sehingga pencarian tidak dilanjutkan. Jika seluruh data telah diperiksa dan target tidak ditemukan, program menampilkan pesan bahwa target tidak ditemukan. Dengan demikian, break berfungsi untuk menghentikan proses pencarian ketika target sudah ditemukan.

Tugas 10 – continue

Algoritma

1. Mulai.
2. Deklarasikan variabel *i* bertipe integer.
3. Lakukan perulangan dari *i* = 1 sampai *i* = 10.
4. Periksa apakah *i* merupakan bilangan genap dengan kondisi $i \% 2 == 0$.
5. Jika kondisi benar, gunakan continue untuk melewati proses menampilkan angka tersebut.
6. Jika kondisi salah, tampilkan nilai *i*.
7. Ulangi langkah 4–6 sampai ***i* = 10**.
8. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main() {
4      int i;
5
6      printf("Data angka 1 sampai 10:\n");
7
8      for (i = 1; i <= 10; i++) {
9          if (i % 2 == 0) {
10             continue;
11         }
12
13         printf("%d ", i);
14     }
15
16     return 0;
17 }
```

Output

```
Data angka 1 sampai 10:
1 3 5 7 9
```

Analisis

Pada program tersebut, continue digunakan untuk melewati angka genap. Kondisinya adalah: $i \% 2 == 0$. Jika angka habis dibagi 2, berarti angka tersebut genap, sehingga continue dijalankan. Akibatnya, perintah printf() dilewati dan program langsung melanjutkan ke perulangan berikutnya.

5.8 TANTANGAN PENGEMBANGAN

1. Triangular

Algoritma

1. Mulai.
2. Deklarasikan variabel n, i, dan jumlah.
3. Masukkan nilai n.
4. Inisialisasi jumlah = 0.
5. Lakukan perulangan i dari 1 sampai n.
6. Tambahkan nilai i ke dalam jumlah.
7. Setelah perulangan selesai, tampilkan nilai jumlah.
8. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main() {
4      int n, i, jumlah = 0;
5
6      printf("Masukkan nilai n: ");
7      scanf("%d", &n);
8
9      for (i = 1; i <= n; i++) {
10         jumlah = jumlah + i;
11     }
12
13     printf("Jumlah triangular = %d\n", jumlah);
14
15     return 0;
16 }
```

Output

```
Masukkan nilai n: 10
Jumlah triangular = 55
```

Analisis

Program digunakan untuk menghitung jumlah triangular dari 1 sampai n. Perulangan for digunakan untuk menjumlahkan setiap bilangan mulai dari 1 hingga nilai n. Variabel jumlah digunakan untuk menyimpan hasil penjumlahan sementara.

2. Menu Berulang

Algoritma

1. Mulai.
2. Deklarasikan variabel pilihan.
3. Tampilkan menu pilihan.
4. Masukkan pilihan dari pengguna.
5. Periksa pilihan pengguna.
6. Jika pilihan 1, tampilkan pilihan satu.
7. Jika pilihan 2, tampilkan pilihan dua.
8. Jika pilihan 3, tampilkan pilihan tiga.
9. Jika pilihan 0, tampilkan pesan program selesai.
10. Jika pilihan selain 0–3, tampilkan pesan pilihan tidak tersedia.
11. Ulangi menu selama pilihan bukan 0.
12. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main() {
4      int pilihan;
5
6      do {
7          printf("\n===== MENU =====\n");
8          printf("1. Pilihan Satu\n");
9          printf("2. Pilihan Dua\n");
10         printf("3. Pilihan Tiga\n");
11         printf("0. Keluar\n");
12         printf("Pilih menu: ");
13         scanf("%d", &pilihan);
14
15         if (pilihan == 1) {
16             printf("Anda memilih Pilihan Satu\n");
17         } else if (pilihan == 2) {
18             printf("Anda memilih Pilihan Dua\n");
19         } else if (pilihan == 3) {
20             printf("Anda memilih Pilihan Tiga\n");
21         } else if (pilihan == 0) {
22             printf("Program selesai.\n");
23         } else {
24             printf("Pilihan tidak tersedia.\n");
25         }
26
27     } while (pilihan != 0);
28
29     return 0;
30 }
```

Output

```
===== MENU =====
1. Pilihan Satu
2. Pilihan Dua
3. Pilihan Tiga
0. Keluar
Pilih menu: 2
Anda memilih Pilihan Dua

===== MENU =====
1. Pilihan Satu
2. Pilihan Dua
3. Pilihan Tiga
0. Keluar
Pilih menu: 3
Anda memilih Pilihan Tiga

===== MENU =====
1. Pilihan Satu
2. Pilihan Dua
3. Pilihan Tiga
0. Keluar
Pilih menu: 0
Program selesai.
```

Analisis

Program menggunakan perulangan do-while untuk membuat menu dapat digunakan berulang kali. Menu akan selalu ditampilkan setidaknya satu kali karena perintah dalam do dijalankan terlebih dahulu sebelum kondisi diperiksa. Program akan terus mengulang selama pengguna tidak memilih 0. Ketika pengguna memilih 0, kondisi pilihan $\neq 0$ menjadi salah sehingga perulangan berhenti.

3. Input Nilai

Algoritma

1. Mulai.
2. Deklarasikan variabel nilai, jumlah, banyak, dan rataRata.
3. Inisialisasi jumlah = 0 dan banyak = 0.
4. Masukkan nilai dari pengguna.
5. Periksa apakah nilai sama dengan -1.
6. Jika nilai bukan -1, tambahkan nilai ke jumlah dan tambah banyak dengan 1.
7. Jika nilai -1, hentikan proses input.
8. Hitung rata-rata dengan membagi jumlah dengan banyak.
9. Tampilkan rata-rata.
10. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main() {
4      float nilai, jumlah = 0, rataRata;
5      int banyak = 0;
6
7      do {
8          printf("Masukkan nilai (-1 untuk berhenti): ");
9          scanf("%f", &nilai);
10
11         if (nilai != -1) {
12             jumlah = jumlah + nilai;
13             banyak++;
14         }
15     } while (nilai != -1);
16
17     if (banyak > 0) {
18         rataRata = jumlah / banyak;
19         printf("Rata-rata = %.2f\n", rataRata);
20     } else {
21         printf("Tidak ada nilai yang dimasukkan.\n");
22     }
23
24     return 0;
25 }
26
```

Output

```
Masukkan nilai (-1 untuk berhenti): 10
Masukkan nilai (-1 untuk berhenti): 20
Masukkan nilai (-1 untuk berhenti): 30
Masukkan nilai (-1 untuk berhenti): -1
Rata-rata = 20.00
```

Analisis

Program digunakan untuk menerima nilai secara berulang dan menghitung rata-ratanya. Nilai -1 digunakan sebagai tanda berhenti sehingga tidak dianggap sebagai nilai siswa. Oleh karena itu, ketika pengguna memasukkan nilai selain -1, nilai tersebut ditambahkan ke jumlah dan jumlah data (banyak) bertambah satu. Setelah pengguna memasukkan -1, program menghentikan input dan menghitung rata-rata dari nilai yang telah dimasukkan.

4. Pola dengan Nested Loop

a. Segitiga Kiri

Algoritma

1. Mulai.
2. Masukkan ukuran segitiga n.

3. Lakukan perulangan baris dari 1 sampai n.
4. Untuk setiap baris, lakukan perulangan kolom dari 1 sampai nilai baris.
5. Tampilkan karakter *.
6. Setelah satu baris selesai, pindah ke baris berikutnya.
7. Ulangi sampai baris ke-n.
8. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main() {
4      int i, j, n;
5
6      printf("Masukkan ukuran: ");
7      scanf("%d", &n);
8
9      for (i = 1; i <= n; i++) {
10         for (j = 1; j <= i; j++) {
11             printf("* ");
12         }
13         printf("\n");
14     }
15
16     return 0;
17 }
```

Output

```
Masukkan ukuran: 3
*
* *
* * *
```

Analisis

Program menggunakan nested loop, yaitu perulangan for di dalam for. Perulangan pertama menentukan jumlah baris, sedangkan perulangan kedua menentukan jumlah * pada setiap baris. Karena jumlah * mengikuti nomor baris, pola yang terbentuk semakin bertambah dari atas ke bawah.

b. Segitiga Terbalik

Algoritma

1. Mulai.
2. Deklarasikan variabel pilihan.
3. Tampilkan menu pilihan.
4. Masukkan pilihan dari pengguna.
5. Periksa pilihan pengguna.
6. Jika pilihan 1, tampilkan pilihan satu.
7. Jika pilihan 2, tampilkan pilihan dua.
8. Jika pilihan 3, tampilkan pilihan tiga.
9. Jika pilihan 0, tampilkan pesan program selesai.

10. Jika pilihan selain 0–3, tampilkan pesan pilihan tidak tersedia.
11. Ulangi menu selama pilihan bukan 0.
12. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main() {
4      int i, j, n;
5
6      printf("Masukkan ukuran: ");
7      scanf("%d", &n);
8
9      for (i = n; i >= 1; i--) {
10         for (j = 1; j <= i; j++) {
11             printf("* ");
12         }
13         printf("\n");
14     }
15
16     return 0;
17 }
```

Output

```
Masukkan ukuran: 4
* * * *
* * *
* *
*
```

Analisis

Program menggunakan nested loop untuk membuat pola segitiga terbalik. Perulangan luar dimulai dari nilai n kemudian berkurang hingga 1. Perulangan dalam mencetak * sebanyak nilai dari perulangan luar. Akibatnya, jumlah * semakin sedikit pada setiap baris.

c. Persegi

Algoritma

1. Mulai.
2. Masukkan ukuran persegi n.
3. Lakukan perulangan baris dari 1 sampai n.
4. Untuk setiap baris, lakukan perulangan kolom dari 1 sampai n.
5. Tampilkan karakter *.
6. Setelah jumlah kolom mencapai n, pindah ke baris berikutnya.
7. Ulangi sampai jumlah baris mencapai n.
8. Selesai.

Source Code

```
1  #include <stdio.h>
2
3  int main() {
4      int i, j, n;
5
6      printf("Masukkan ukuran: ");
7      scanf("%d", &n);
8
9      for (i = 1; i <= n; i++) {
10         for (j = 1; j <= n; j++) {
11             printf("* ");
12         }
13         printf("\n");
14     }
15
16     return 0;
17 }
```

Output

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Analisis

Program menggunakan nested loop, yaitu perulangan for di dalam for. Perulangan pertama menentukan jumlah baris, sedangkan perulangan kedua menentukan jumlah * pada setiap baris. Karena jumlah * mengikuti nomor baris, pola yang terbentuk semakin bertambah dari atas ke bawah.

5.9 ANALISIS DAN REFLEKSI

- 1 Apa perbedaan utama posisi pemeriksaan kondisi pada for, while, dan do-while? for dan while memeriksa kondisi sebelum menjalankan badan loop. do-while memeriksa kondisi setelah menjalankan badan loop.
- 2 Mengapa while dan for dapat tidak menjalankan badan loop sama sekali? Karena kondisi diperiksa terlebih dahulu. Jika kondisi dari awal bernilai **false**, program langsung melewati badan loop tanpa menjalankannya.
- 3 Berikan contoh kasus nyata yang lebih sesuai menggunakan do-while daripada while. Menu program. Menu harus ditampilkan minimal satu kali agar pengguna dapat memilih pilihan. Setelah itu, program mengecek apakah pengguna ingin mengulang atau keluar.
- 4 Apa risiko perulangan tak hingga dan bagaimana cara mencegahnya? Perulangan tak hingga dapat membuat program terus berjalan, menggunakan sumber daya komputer, atau

menyebabkan program tidak merespons. Cara mencegahnya adalah memastikan kondisi loop dapat berubah menjadi false dan variabel pengendali diperbarui dengan benar.

- 5 Jelaskan perbedaan fungsi `break` dan `continue`. `break` digunakan untuk menghentikan seluruh perulangan dan keluar dari loop. `continue` digunakan untuk melewati sisa kode pada iterasi saat ini, kemudian melanjutkan ke iterasi berikutnya.
- 6 Pada `nested loop`, variabel mana yang mengendalikan baris dan mana yang mengendalikan kolom? Biasanya loop luar mengendalikan baris, sedangkan loop dalam mengendalikan kolom. Jadi, setiap satu baris diproses oleh loop luar, kemudian loop dalam mengisi kolom-kolom pada baris tersebut.

KESIMPULAN

Berdasarkan hasil praktikum, dapat disimpulkan bahwa looping merupakan konsep perulangan dalam pemrograman yang digunakan untuk menjalankan suatu perintah secara berulang sesuai dengan kondisi yang telah ditentukan. Pada bahasa C terdapat beberapa jenis looping, yaitu `for`, `while`, dan `do-while`. `for` cocok digunakan ketika jumlah perulangan sudah diketahui, sedangkan `while` digunakan ketika perulangan bergantung pada suatu kondisi. Sementara itu, `do-while` akan menjalankan perintah minimal satu kali karena pemeriksaan kondisi dilakukan setelah badan perulangan dijalankan.

Praktikum juga menunjukkan bahwa `nested loop` dapat digunakan untuk membuat perulangan di dalam perulangan, misalnya untuk mengatur baris dan kolom. Selain itu, `break` digunakan untuk menghentikan perulangan secara langsung, sedangkan `continue` digunakan untuk melewati satu iterasi dan melanjutkan ke iterasi berikutnya. Berdasarkan hasil pengujian program, seluruh konsep perulangan dapat diterapkan sesuai dengan kondisi yang diberikan dan menghasilkan keluaran sesuai tujuan program. Dengan demikian, pemahaman mengenai looping, `nested loop`, `break`, dan `continue` sangat penting untuk membuat program yang lebih terstruktur dan efisien.